

Faculty of Information Technology (FIT) – Ho Chi Minh City University of Science (HCMUS)

CS423 / CSC13003 – Software Testing (AI-augmented · 2026)

AI POLICY · TEMPLATES — 2026 v1.0

AI Audit Report — 5-section Template per Artifact

Mandatory appendix for every AI-assisted homework (HW#01–HW#06, and Seminar).

Adapted from Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0. This adaptation is prepared for FIT@HCMUS – CS423 / CSC15003 Software Testing course.

1. Student Information

Field	Value
Student name (printed):	Cao Trần Bá Đạt
Student ID:	23127168
Class / Cohort:	23KTPM1
Assignment ID (e.g., HW#00, HW#02):	HW#03
Assignment date:	16/08/2026
AI tool(s) used:	Claude
AI tool(s) used:	☒ Yes ☐ No

2. Instructions (read before filling)

- Add one row per AI-generated artifact (test case, script, checklist, OpenAPI spec, JMeter plan, etc.).
- Paste the verbatim prompt — DO NOT paraphrase.
- Paste the verbatim AI output (or include a labelled screenshot in the report).
- Tag the verdict: VALID / INVALID / INCOMPLETE.
- Reasoning must cite a course slide, ISTQB section, or technical RFC.
- Show the corrected artifact with the change highlighted.
- Sample rows are in italic — replace them before submission.

3. Audit Table — one row per artifact

Name of the AI tool	Date and time	Your prompt	The AI output
Claude	18:45 15/08/2026	I am performing API testing on the EShop system (Node.js/Express + SQLite backend). I've attached the README file and the Eshop API specification. We are focusing on testing: Target Feature: FR-06 Product Detail Target Endpoint: GET /api/products/:id Expected Success Response Sample (200 OK): { "id": 1, "name": "iPhone 15 Pro Max", "price": 30000000, "description": "Điện thoại cao cấp của Apple",	EP & BVA Analysis — GET /api/products/:id (FR-06 Product Detail) 0. Scope of the endpoint (per spec) Only one input surface exists for this endpoint: the path parameter id. Unlike GET /api/products (which supports ?search=), the spec shows no query parameters for the detail endpoint, and GET requests carry no body. Section 3.2 is not listed under an "Authorization required" note (unlike Sections 2 and 4), so per the document, this endpoint is presumed public — auth-related partitions (401/403) are included below only as clarification-needed items, not confirmed business rules. A. Parameter: id (path parameter) A.1 Data Type / Format — Equivalence Partitions #PartitionExampleValid/Invalid1Positive integer, well-formed1,

Name of the AI tool	Date and time	Your prompt	The AI output
		"imageUrl": "https://placeholder.co/300x300/png?text=iPhone+15", "category_id": 1 } Expected Error Response Formats (400 / 401 / 403 / 404 / 422): { "error": "string" } or { "error": "string", "errors": [] } Your task: Analyze the business rules, data types, query/path/body parameters, and constraints for this target API based strictly on the provided documents. Apply Equivalence Partitioning (EP) and Boundary Value Analysis (BVA) on all parameters. Identify all Valid and Invalid partitions (data types, formats, lengths, positive/negative bounds, non-existent references) List these partitions clearly in a structured list. Do not generate the test case table yet. Attach README.md, api_specification.md	25Valid2Zero0Invalid (IDs assumed to start at 1)3Negative integer-1Invalid4Decimal / float1.5Invalid5Scientific notation1e5Invalid6Non-numeric stringabc, iPhoneInvalid7Alphanumeric mix12abcInvalid8Leading zeros0001Invalid/ambiguous (flag — may or may not be normalized)9Hex/octal-looking value0x1A, 017Invalid10Unicode/full-width digits 1 2 3 Invalid11Whitespace only / blank" "Invalid12Empty segment (trailing slash)/api/products/Invalid — resolves to a different route (list), not this endpoint13Literal null-like tokensnull, undefined, NaNInvalid14Boolean-like tokentrueInvalidA.2 Security-adjacent Format Partitions (still EP, invalid class) #PartitionExample15SQL injection payload1 OR 1=1, 1' OR '1'='116XSS payload<script>alert(1)</script>17Path traversal payload../etc/passwd18URL-encoded/special chars%20, %00, #, ;, ,A.3 Boundary Value Analysis — numeric id Spec gives no explicit min/max, so boundaries are derived from (a) the implied auto-increment PK starting at 1, and (b) the actual max existing product ID in the seeded/test DB (call it N). Lower boundary: id = -1 → below min, invalid id = 0 → boundary, invalid (no product 0 expected) id = 1 → minimum valid boundary (first product, if it exists) id = 2 → min + 1, valid Upper boundary (relative to dataset): id = N → max existing product → valid, 200 id = N - 1 → valid, 200 id = N + 1 → just above max, syntactically valid but non-existent → 404 Upper boundary (relative to data-type/storage limits): id = 2147483647 (INT32 max) → likely 404, but risk of type-handling error id = 2147483648 (INT32 max + 1) → potential overflow behavior (500 vs 404) — flag as risk area given SQLite backend id = 9223372036854775807 (INT64/SQLite max) → edge overflow case id = 9999999999999999999 (exceeds even 64-bit / JS safe integer) → parsing/overflow edge, likely invalid A.4 Business / Existence Partitions Valid: id is a well-formed positive integer and exists in the Products table → 200, response body matches full product schema (id, name, price, description, imageUrl, category_id) Invalid: id is well-formed but does not exist in DB → 404 Invalid (unconfirmed by spec): id exists but product is soft-deleted/inactive, if such a state exists — not documented, should be flagged as an assumption/open question to the dev team rather than assumed Invalid (unconfirmed by spec): id exists but its category_id no longer exists (orphaned FK) — data-integrity edge case, not directly testable via inputs to this endpoint alone B. Header Dimension (Authorization) Per spec, Section 3.1/3.2 (product read endpoints) carry no "Authorization required" note, unlike Sections 2 and 4. Valid: No Authorization header sent → expect normal 200/404 behavior (public read) Valid: Authorization header present with a valid token → should not change behavior Flag for clarification (not a confirmed EP per doc): Invalid/expired/malformed token sent — since the endpoint isn't

Name of the AI tool	Date and time	Your prompt	The AI output
			documented as protected, a 401/403 here would be unexpected; this partition should be tested to confirm the API does not mistakenly enforce auth on a public GET, rather than to validate an auth rule that doesn't exist in the spec. C. Out-of-scope / Not Applicable Dimensions Query parameters: Not documented for :id detail endpoint (only the list endpoint supports ?search=) — no EP/BVA needed here, though appending an unexpected query string (e.g., /api/products/1?foo=bar) could be tested as a robustness edge case (should be ignored, still 200). Request body: Not applicable — GET request, spec defines no body for this endpoint. Route-level malformed paths: e.g., /api/products/1/1, /api/products//1 — these are routing-layer edge cases (404 route-not-found vs. application-level 404) worth noting separately from the id parameter partitions above, since they test the web framework's routing rather than the endpoint's input validation. Summary count Valid partitions: 1 (format) × existence-confirmed → 200 path Invalid partitions: ~18 format/type classes + 2 existence classes + BVA edge points (0, -1, N+1, INT overflow tier) + 2 open/flagged assumptions (soft-delete state, auth enforcement check) Let me know when you'd like me to convert these partitions into the actual test case table (Test ID, Precondition, Input, Expected Result, Expected HTTP Status/Error Format).
Claude	18:54 15/08/2026	EP & BVA Analysis — GET /api/products/:id (FR-06 Product Detail) 1. Phân tích đặc tả & Ràng buộc: Target Feature: FR-06 Product Detail; Target Endpoint: GET /api/products/:id; Method: HTTP GET (no body); Input Surface: Path parameter :id (no query params); Auth: Public read (không bắt buộc token); Kiểu dữ liệu  Primary Key, Integer, auto-increment (id >= 1); Success Response (200 OK): {"id": 1, "name": "iPhone 15 Pro Max", "price": 30000000, "description": "Điện thoại cao cấp của Apple", "imageUrl": "https://placeholder.co/300x300/png?text=iPhone+15", "category_id": 1}; Error Formats (400/404/422): {"error": "string"} hoặc {"error": "string", "errors": []}. 2. Phân vùng tương đương (Equivalence Partitioning - EP): [Valid Partitions]: Số nguyên dương chuẩn (id=1, 25); ID tồn tại trong DB -> 200 OK. [Invalid Partitions - Kiểu số]: id=0;	# EShop — GET /api/products/:id — Full Test Suite Assumptions (stated explicitly since not in spec): - DB is seeded with at least product id=1 (iPhone 15 Pro Max, matches the sample response) and a known maximum existing ID, assumed id=50 (N=50) for boundary tests. Replace with actual seed data before execution. - A known non-existent ID is assumed to be 99999. - Endpoint is public per spec (no Authorization note in Section 3.2), so auth-related tests are framed as confirming no unintended enforcement, not validating a documented rule. - imageUrl is expected as a well-formed URL string; price/category_id expected as integers per sample. [TEST CASES]: TC-01 (Domain): Get existing product by valid ID Pre: Product id=1 exists in DB GET /api/products/1 Headers: (none) Body: — Status: 200 Expected: JSON object with exactly keys: id, name, price, description, imageUrl, category_id; id==1; price > 0. TC-02 (Boundary): Get product at ID = min + 1 Pre: Product id=2 exists in DB GET /api/products/2 Headers: (none) Body: — Status: 200 Expected: id==2; matches full schema; all fields non-null. TC-03 (Boundary): Get product at max existing ID (N) Pre: Product id=50 exists GET /api/products/50 Headers: (none) Body: — Status: 200 Expected: id==50; schema valid; category_id >= 1. TC-04 (Boundary): Get product at N — 1 Pre: Product id=49 exists GET /api/products/49 Headers: (none) Body: — Status: 200 Expected: id==49; schema valid. TC-05 (Boundary): Get product at N + 1 (non-existent) Pre: No

Name of the AI tool	Date and time	Your prompt	The AI output
		Số âm (id=-1, -25); Số thập phân (id=1.5); Ký hiệu khoa học (id=1e5); Có số 0 ở đầu (id=0001); Hex/Octal (id=0x1A, 017); Chữ số full-width Unicode (id= 1 2 3). [Invalid Partitions - Phi số & Chuỗi]: Chuỗi chữ cái (id=abc); Chữ và số kết hợp (id=12abc); Khoảng trắng (%20); Literal tokens (null, undefined, NaN, true, false); Ký tự đặc biệt (%00, @, #). [Security Partitions]: SQLi (1 OR 1=1, 1' OR '1'='1, 1; DROP TABLE products;--); XSS (<script>alert(1) </script>); Path Traversal (../etc/passwd). [Business Existence]: ID đúng định dạng nhưng không tồn tại trong DB (id=99999) -> 404 Not Found. 3. Phân tích giá trị biên (Boundary Value Analysis - BVA) với N là ID max hiện có: [Biên dưới]: id=-1 (Dưới biên min, Invalid); id=0 (Điểm chia âm/dương, Invalid); id=1 (Biên hợp lệ nhỏ nhất, Valid 200); id=2 (Cận trên biên min, Valid 200). [Biên trên theo DB]: id=N-1 (Valid 200); id=N (Biên max hiện có, Valid 200); id=N+1 (Vượt quá max hiện có, Invalid/Non-existent 404). [Biên trên theo giới hạn kiểu dữ liệu]: id=2147483647 (INT32 Max, 404); id=2147483648 (INT32 Overflow, 400/404); id=9223372036854775807 (SQLite INT64 Max, 400/404); id=9999999999999999999 (Vượt quá JS Safe Int, 400/404).	product id=51 GET /api/products/51 Headers: (none) Body: — Status: 404 Expected: {"error": "<string>". TC-06 (Boundary): Get product at lower bound ID = 0 Pre: No product id=0 GET /api/products/0 Headers: (none) Body: — Status: 400 / 404 Expected: {"error": "<string>". TC-07 (Boundary): Negative ID value Pre: No product id=-1 GET /api/products/-1 Headers: (none) Body: — Status: 400 / 404 Expected: {"error": "<string>". TC-08 (Domain): Non-existent positive integer ID Pre: No product id=99999 GET /api/products/99999 Headers: (none) Body: — Status: 404 Expected: {"error": "<string>". TC-09 (Boundary): ID at INT32 max ($2^{31} - 1$) Pre: No product id=2147483647 GET /api/products/2147483647 Headers: (none) Body: — Status: 404 Expected: {"error": "<string>"; Response time < 2s. TC-10 (Boundary): ID at INT32 overflow (2^{31}) Pre: No product id=2147483648 GET /api/products/2147483648 Headers: (none) Body: — Status: 400 / 404 Expected: {"error": "<string>"; Must not return 500. TC-11 (Boundary): Extreme overflow beyond 64-bit Pre: — GET /api/products/9999999999999999999 Headers: (none) Body: — Status: 400 / 404 Expected: {"error": "<string>"; No stack trace leak. TC-12 (Domain): Decimal float ID Pre: — GET /api/products/1.5 Headers: (none) Body: — Status: 400 Expected: {"error": "<string>". TC-13 (Domain): Non-numeric alphabetic ID Pre: — GET /api/products/abc Headers: (none) Body: — Status: 400 Expected: {"error": "<string>". TC-14 (Domain): Alphanumeric mixed ID Pre: — GET /api/products/12abc Headers: (none) Body: — Status: 400 Expected: {"error": "<string>". TC-15 (Domain): Leading zeros ID Pre: Product id=1 exists GET /api/products/0001 Headers: (none) Body: — Status: 200 / 400 Expected: id==1 if normalized, else 400 Bad Request. TC-16 (Domain): Scientific notation ID Pre: — GET /api/products/1e5 Headers: (none) Body: — Status: 400 Expected: {"error": "<string>". TC-17 (Domain): Hexadecimal representation ID Pre: — GET /api/products/0x1A Headers: (none) Body: — Status: 400 Expected: {"error": "<string>". TC-18 (Domain): Full-width Unicode digit ID Pre: — GET /api/products/ 1 Headers: (none) Body: — Status: 400 Expected: {"error": "<string>". TC-19 (Domain): Boolean literal string ID Pre: — GET /api/products/true Headers: (none) Body: — Status: 400 Expected: {"error": "<string>". TC-20 (Domain): Null literal string ID Pre: — GET /api/products/null Headers: (none) Body: — Status: 400 Expected: {"error": "<string>". TC-21 (Domain): Whitespace encoded ID Pre: — GET /api/products/%20 Headers: (none) Body: — Status: 400 / 404 Expected: {"error": "<string>". TC-22 (Robustness): Missing ID segment (trailing slash) Pre: Products

Name of the AI tool	Date and time	Your prompt	The AI output
			exist in DB GET <code>/api/products/</code> Headers: (none) Body: — Status: 200 Expected: Returns JSON Array (resolves to list API). TC-23 (Robustness): Double-slash route path Pre: — GET <code>/api/products//1</code> Headers: (none) Body: — Status: 400 / 404 Expected: <code>{"error": "<string>"}</code>; No 500 error. TC-24 (Robustness): Nested sub-path after ID Pre: Product id=1 exists GET <code>/api/products/1/1</code> Headers: (none) Body: — Status: 404 Expected: <code>{"error": "<string>"}</code> (Unmatched route). TC-25 (Robustness): Unexpected query string ignored Pre: Product id=1 exists GET <code>/api/products/1?foo=bar</code> Headers: (none) Body: — Status: 200 Expected: Query string ignored; body identical to TC-01. TC-26 (Security): SQLi Boolean-based payload in <code>:id</code> Pre: — GET <code>/api/products/1%20OR%201=1</code> Headers: (none) Body: — Status: 400 Expected: <code>{"error": "<string>"}</code>; No full table dump. TC-27 (Security): SQLi Quote-break payload in <code>:id</code> Pre: — GET <code>/api/products/1'%20OR%20'1'='1</code> Headers: (none) Body: — Status: 400 Expected: <code>{"error": "<string>"}</code>; No SQLite error exposed. TC-28 (Security): SQLi Stacked query / comment payload Pre: — GET <code>/api/products/1;--</code> Headers: (none) Body: — Status: 400 Expected: <code>{"error": "<string>"}</code>; DB state unmodified. TC-29 (Security): Reflected XSS script tag in <code>:id</code> Pre: — GET <code>/api/products/%3Cscript%3Ealert(1)%3C/script%3E</code> Headers: (none) Body: — Status: 400 Expected: <code>Content-Type: application/json</code>; Raw <code><script></code> not reflected. TC-30 (Security): Directory Traversal payload Pre: — GET <code>/api/products/../../../../etc/passwd</code> Headers: (none) Body: — Status: 400 / 404 Expected: <code>{"error": "<string>"}</code>; No system file exposed. TC-31 (Security): Public endpoint auth check Pre: Product id=1 exists GET <code>/api/products/1</code> Headers: (no Auth header) Body: — Status: 200 Expected: Returns product schema; NOT 401. TC-32 (Security): Ignore invalid Authorization header Pre: Product id=1 exists GET <code>/api/products/1</code> Headers: <code>Authorization: Bearer invalid.token</code> Body: — Status: 200 Expected: Ignores token on public endpoint; returns 200. TC-33 (Security): HTTP Verb Tampering (Unauthorized Delete) Pre: Product id=1 exists DELETE <code>/api/products/1</code> Headers: (no Admin token) Body: — Status: 401 / 403 Expected: <code>{"error": "<string>"}</code>. TC-34 (Schema): Full schema & keys presence check Pre: Product id=1 exists GET <code>/api/products/1</code> Headers: (none) Body: — Status: 200 Expected: Exactly 6 keys: <code>id</code>, <code>name</code>, <code>price</code>, <code>description</code>, <code>imageUrl</code>, <code>category_id</code>. TC-35 (Schema): Error schema structure on 404 Pre: No product id=99999 GET <code>/api/products/99999</code> Headers: (none) Body: — Status: 404 Expected: JSON object with <code>error</code> (string); No product keys. TC-36 (Schema): Validate <code>price</code> field type and bound Pre: Product id=1 exists GET <code>/api/products/1</code> Headers: (none)

Name of the AI tool	Date and time	Your prompt	The AI output
			Body: — Status: 200 Expected: <code>typeof price === "number"</code> and <code>price > 0</code> . Notes for execution: * TC-03, TC-04, TC-05 require confirming the actual maximum seeded ID (<i>N</i>) in the database before running — replace <code>50</code> , <code>49</code> , and <code>51</code> with the real values from your test database. * TC-06, TC-07, TC-10, TC-11, TC-15, TC-21, TC-23 have ambiguous expected behavior per the specification (<code>400</code> vs <code>404</code> are not clearly distinguished between malformed input and non-existent resource misses) — flag these as clarification items for the development team; both acceptable status codes are listed so the tester can record actual vs. expected results and file a specification-gap defect if inconsistent. * TC-31 to TC-33 assume products are catalog-wide public resources (not user-owned) per Section 3.3; if future specification revisions introduce per-seller resource scoping, the IDOR testing boundary should be re-evaluated. * All security test cases (TC-26 to TC-30) should also assert that the response latency stays consistent with normal <code>400/404</code> responses to detect potential blind or time-based injection vulnerabilities.
Claude	15:19 16/08/2026	I am performing API testing on the EShop system (Node.js/Express + SQLite backend) based strictly on the EShop API Specification. Target Feature: FR-08 Checkout (Đặt hàng) Target Endpoint: POST /api/checkout Required Header: Authorization: Bearer Expected Request Body (JSON): { "total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM" } Expected Success Response Sample (200 OK): JSON containing order confirmation/details Expected Error Response Formats (400 Bad Request / 401 Unauthorized / 404 Not Found / 422 Unprocessable): Standard JSON error response YOUR TASK: Business & Parameter Analysis:Analyze the business rules, authentication requirements, and constraints for total_amount (numeric, positive, non-zero, float/int bounds) and shipping_address (string, length, non-empty, special characters, unicode). Apply Equivalence Partitioning (EP) and Boundary Value Analysis (BVA) on all request body parameters and authorization	EShop — POST /api/checkout — Full Test Suite (FR-08) Global assumptions (stated explicitly, not in spec): Test user: test@domain.com / Password123! (registered via POST /api/register if not already existing). Standard cart setup = POST /api/cart with {"id":1,"name":"Sản phẩm A","price":100000,"quantity":2} → cart total = 200000, matching the sample total_amount. A valid JWT is obtained via POST /api/login and reused as <VALID_TOKEN> placeholder only inside Pre-request Script (concrete at runtime) — request bodies themselves always show concrete literal values per the no-placeholder rule. Per instructions, error assertions check status code + Content-Type: application/json only, never exact error strings. Test IDCategoryTest DescriptionPre-conditionsHTTP Method & URLRequest BodyPre-request ScriptExpected HTTP StatusExpected Response Body / JSON Schema AssertionActual ResultVerdictTC-01Domain / Happy PathValid checkout with matching cart total, ASCII addressUser registered; cart empty initiallyPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM"}Login test@domain.com/Password123! → store token as Bearer header. POST /api/cart with {"id":1,"name":"Sản phẩm A","price":100000,"quantity":2}200Content-Type: application/json; body is object containing order identifier field (e.g. id/order_id) and total_amount==200000TC-02Domain / Happy PathValid checkout with Vietnamese unicode addressSame as TC-01POST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "Số 45, Ngõ 12, Đường Nguyễn Trãi, Phường Thanh Xuân, TP.HCM"}Login + cart setup identical to TC-01200Content-Type: application/json; response shipping_address (if echoed) preserves unicode characters

[illegible]

Name of the AI tool	Date and time	Your prompt	The AI output
		error string assertions (e.g. avoid checking {"error": ""}). Only assert HTTP Status Codes, Content-Type: application/json, and positive schema/data assertions for 200 OK. Leave "Actual Result" and "Verdict" columns empty for test execution recording.	Error; no stack trace leakedTC-12Domaintotal_amount field missing entirelyStandard cart presentPOST http://localhost:3000/api/checkout{"shipping_address": "123 Le Loi, TP.HCM"}Login + standard cart setup400Content-Type: application/json; JSON error objectTC-13Boundaryshipping_address empty stringStandard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": ""}Login + standard cart setup400 or 422Content-Type: application/json; JSON error objectTC-14Boundaryshipping_address whitespace-only stringStandard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": " "}Login + standard cart setup400 or 422Content-Type: application/json; JSON error object (whitespace treated as effectively empty)TC-15Boundaryshipping_address extremely long string (length stress, 5000 chars)Standard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "aaaaaaaaa..." } (literal string of 5000 repeated 'a' characters)Login + standard cart setup400 or 200 (document actual max-length behavior)Content-Type: application/json; response within 3s; NOT 500; if 200 confirm no DB truncation errorTC-16Typeshipping_address as numeric typeStandard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": 12345}Login + standard cart setup400Content-Type: application/json; JSON error object indicating type validation failureTC-17Typeshipping_address as nullStandard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": null}Login + standard cart setup400Content-Type: application/json; JSON error objectTC-18Typeshipping_address as array typeStandard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": ["123 Le Loi, TP.HCM"]}Login + standard cart setup400Content-Type: application/json; JSON error objectTC-19Domainshipping_address field missing entirelyStandard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000}Login + standard cart setup400Content-Type: application/json; JSON error objectTC-20DomainCompletely empty JSON bodyStandard cart presentPOST http://localhost:3000/api/checkout{}Login + standard cart setup400Content-Type: application/json; JSON error object, ideally listing both missing fieldsTC-21Business LogicCheckout attempt with empty cart (no items added)User logged in; cart explicitly empty (no prior POST /api/cart calls, or cart cleared)POST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM"}Login only — deliberately skip cart setup step; optionally verify via GET

Name of the AI tool	Date and time	Your prompt	The AI output
			/api/cart returns empty array first400 or 422Content-Type: application/json; JSON error object indicating empty cart / nothing to checkoutTC-22Business LogicPrice/total tampering — total_amount does not match actual cart sumCart contains item priced 100000 x2 = 200000 actual totalPOST http://localhost:3000/api/checkout{"total_amount": 1, "shipping_address": "123 Le Loi, TP.HCM"}Login + standard cart setup (actual cart total = 200000, submitted total_amount = 1)400 or 422Content-Type: application/json; JSON error object indicating amount mismatch; verify via GET /api/orders/my-orders that no order was created with tampered totalTC-23SecurityMissing Authorization header entirelyStandard cart present (as another authenticated session)POST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM"}Login + cart setup performed, but Authorization header deliberately omitted on this specific request401Content-Type: application/json; JSON error objectTC-24SecurityInvalid/malformed Bearer token (garbage string)—POST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM"}Set header Authorization: Bearer garbage.invalid.token123 (no real login performed)401Content-Type: application/json; JSON error objectTC-25SecurityExpired Bearer tokenA previously-issued token that has passed its expiry time (obtained earlier and allowed to expire, or crafted with past exp claim)POST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM"}Use stored expired token from a prior test session in Authorization: Bearer <EXPIRED_TOKEN>401Content-Type: application/json; JSON error object indicating token expiry/invalidityTC-26SecurityRole/claim tampering — JWT payload modified to elevate role, resulting in invalid signatureValid token obtained via login, then payload segment locally edited (e.g. role changed to admin) without re-signingPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM"}Login → decode JWT → modify payload claim → reassemble token with original (now-invalid) signature → send in Authorization header401Content-Type: application/json; JSON error object; server rejects due to signature mismatch, confirming no role-escalation is possible via client-side token tamperingTC-27SecuritySQL Injection payload in shipping_addressStandard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi"; DROP TABLE orders;--"}Login + standard cart setup200 or 400 (must not corrupt data)Content-Type: application/json; if 200, subsequent GET /api/orders/my-orders still returns valid data (table not dropped); no DB error string in bodyTC-28SecurityXSS payload in

Name of the AI tool	Date and time	Your prompt	The AI output
			shipping_addressStandard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "alert(document.cookie)}Login + standard cart setup200 or 400Content-Type: application/json; if 200, payload stored as inert string (not executed) and any echo of it in response is unescaped-but-inert JSON string, not rendered HTMLTC-29SecuritySQL Injection-style payload in total_amount (type confusion attempt)Standard cart presentPOST http://localhost:3000/api/checkout{"total_amount": "1 OR 1=1", "shipping_address": "123 Le Loi, TP.HCM"}Login + standard cart setup400Content-Type: application/json; JSON error object indicating type validation failure; no unintended bulk data exposureTC-30RobustnessMalformed JSON syntax in request bodyStandard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM" (missing closing brace — sent as raw malformed text)Login + standard cart setup400Content-Type: application/json; JSON error object; NOT 500; no raw parser stack trace in bodyTC-31RobustnessInvalid Content-Type header (text/plain instead of application/json)Standard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM"}Login + standard cart setup; override request header to Content-Type: text/plain before sending400 or 415Content-Type: application/json; JSON error object indicating unsupported media type/parsing failureTC-32RobustnessHTTP verb tampering — GET on checkout routeStandard cart presentGET http://localhost:3000/api/checkout— (GET carries no body)Login (Authorization header set); no body sent404 or 405Content-Type: application/json; JSON error object; route/method not allowedTC-33RobustnessHTTP verb tampering — DELETE on checkout routeStandard cart presentDELETE http://localhost:3000/api/checkout—Login (Authorization header set); no body sent404 or 405Content-Type: application/json; JSON error objectTC-34RobustnessHTTP verb tampering — PUT on checkout routeStandard cart presentPUT http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM"}Login (Authorization header set)404 or 405Content-Type: application/json; JSON error objectTC-35Boundarytotal_amount at JS/JSON-safe integer maximum boundary, matching cartCart contains single item priced 9007199254740991 VND (synthetic test data)POST http://localhost:3000/api/checkout{"total_amount": 9007199254740991, "shipping_address": "123 Le Loi, TP.HCM"}Login. POST /api/cart with {"id":3,"name":"Sản phẩm Test Boundary","price":9007199254740991,"quantity":1}200 or 400 (document actual precision-handling behavior)Content-Type: application/json; if 200, total_amount returned without

Name of the AI tool	Date and time	Your prompt	The AI output
			precision loss; if 400, JSON error objectTC-36Schema / ContractValidate success response schema fully on standard valid checkoutStandard cart presentPOST http://localhost:3000/api/checkout{"total_amount": 200000, "shipping_address": "123 Le Loi, TP.HCM"}Login + standard cart setup200Content-Type: application/json; body is JSON object containing at minimum an order-identifying key (id or order_id, integer), total_amount==200000 (number), shipping_address=="123 Le Loi, TP.HCM" (string) and a status field (e.g. status=="pending") if present; no 500-level fields (no stack trace, no null identifier) Execution notes: TC-04–TC-11, TC-13–TC-20, TC-27–TC-29 all assume the endpoint validates request body before touching cart state — if the API instead evaluates cart/business rules first, some 400-expected cases may return 422 or a business-logic error instead; both codes are listed where ambiguous per spec, and actual behavior should be recorded as a spec-gap note if inconsistent across similar invalid-input cases. TC-21's "empty cart" precondition requires either a fresh user with no cart history, or an explicit cart-clear step — the spec does not document a DELETE /api/cart endpoint, so this may need to be arranged via a new test user account. TC-25 requires either a genuinely time-expired token (test timing dependency) or coordination with backend team to issue a short-TTL token for testing — flag as an environment-setup dependency. TC-26/TC-24 confirm signature validation, not role-based authorization specifically, since the spec does not document role-scoped rules for checkout — noted as an assumption. All 200-path tests (TC-01–TC-03, TC-27/28 optimistic branch, TC-35, TC-36) should be followed by a GET /api/orders/my-orders verification step to confirm the order was actually persisted, recommended as a shared post-condition check during execution rather than a separate test case.
Claude	19:02 16/08/2026	I am performing API testing on the EShop system (Node.js/Express + SQLite backend) based strictly on the EShop API Specification. Target Feature: FR-16 Import Products (Import sản phẩm) Target Endpoint: POST /api/admin/import-products Required Header: Authorization: Bearer <admin_token> Expected Request Body (JSON): { "products": [{ "name": "Sản phẩm A", "price": 100000, "description": "Mô tả", "imageUrl": "http://...", "category_id": 1 }] } Expected Success Response Sample (200 OK): JSON containing import confirmation and imported	EShop — POST /api/admin/import-products — Full Test Suite (FR-16) Global assumptions (stated explicitly, not in spec): Admin account: admin@eshop.com / Admin123! (assumed pre-seeded with admin role; not creatable via public /api/register). Regular user account: test@domain.com / Password123! (standard non-admin). A known non-existent category_id is assumed to be 99999. A known existing category is assumed category_id=1 (via GET /api/categories). Per instructions, error assertions check status code + Content-Type: application/json + generic error-object shape only — never exact error strings. Test IDCategoryTest DescriptionPre-conditionsHTTP Method & URLRequest BodyPre-request ScriptExpected HTTP StatusExpected Response Body / JSON Schema AssertionActual ResultVerdictTC-01Domain / Happy PathValid import of a single well-formed productAdmin account exists; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "Sản phẩm A", "price": 100000,

Name of the AI tool	Date and time	Your prompt	The AI output
		count/details Expected Error Response Formats (400 Bad Request / 401 Unauthorized / 403 Forbidden / 404 Not Found / 422 Unprocessable): Standard JSON error response YOUR TASK: Business & Parameter Analysis: Analyze business rules, Admin authentication/RBAC constraints, array requirements for products, and field rules for name (string, non-empty, unicode), price (numeric, positive, non-zero), description, imageUrl, and category_id (valid foreign key). Apply Equivalence Partitioning (EP) and Boundary Value Analysis (BVA) on all request body parameters, product array items, and authorization headers. Comprehensive Test Dimension Coverage: Domain & Happy Path (Valid single/bulk product import, unicode text); Boundary Value Analysis (BVA) & Type Domain (Empty array [], empty object {}, missing fields, price <= 0/float/string/null types, extreme numbers, invalid category_id, long strings); Business Logic & Integrity (Duplicate names, batch atomicity/partial failures); Security Testing (Missing token, invalid token, regular user privilege escalation, SQL Injection & XSS payloads in name/description); Robustness & Protocol Testing (Malformed JSON, invalid Content-Type, Mass Assignment extra fields, HTTP verb tampering like GET/DELETE/PUT). Test Suite Generation: Synthesize all scenarios into a single, comprehensive Test Suite containing EXACTLY 36 distinct, fully-detailed test cases (numbered TC-01 to TC-36). OUTPUT FORMAT: Output the result strictly as a clean Markdown table with the following 11	<pre>"description": "Mô tả", "imageUrl": "http://example.com/a.png", "category_id": 1}}Login admin@eshop.com/Admin123! → store token as Authorization: Bearer 200Content-Type: application/json; body contains an imported-count field (e.g. imported/count==1) or array of created records with id assignedTC-02Domain / Happy PathValid bulk import of multiple products (3 items)Admin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name":"SP1","price":10000,"description":"Mô tả 1","imageUrl":"http://example.com/1.png","category_id":1}, {"name":"SP2","price":20000,"description":"Mô tả 2","imageUrl":"http://example.com/2.png","category_id":1}, {"name":"SP3","price":30000,"description":"Mô tả 3","imageUrl":"http://example.com/3.png","category_id":1}}Login admin@eshop.com/Admin123!200Content-Type: application/json; imported count == 3; each item retrievable afterward via GET /api/productsTC-03Domain / Happy PathValid import with Vietnamese unicode in name/descriptionAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "Điện thoại Samsung Galaxy Ω", "price": 15000000, "description": "Điện thoại cao cấp — hàng chính hãng, bảo hành 12 tháng", "imageUrl": "http://example.com/galaxy.png", "category_id": 1}}Login admin@eshop.com/Admin123!200Content-Type: application/json; imported product's name/description preserved with unicode characters intact (no mojibake/escaping loss)TC-04BoundaryEmpty products arrayAdmin logged inPOST http://localhost:3000/api/admin/import-products{"products": []}}Login admin@eshop.com/Admin123!400 or 422Content-Type: application/json; JSON error object indicating no products to importTC-05BoundaryCompletely empty JSON bodyAdmin logged inPOST http://localhost:3000/api/admin/import- products{}Login admin@eshop.com/Admin123!400Content- Type: application/json; JSON error object indicating missing products fieldTC-06DomainMissing "products" key entirely (unrelated key sent instead)Admin logged inPOST http://localhost:3000/api/admin/import-products{"items": [{"name": "SP1", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}}Login admin@eshop.com/Admin123!400Content-Type: application/json; JSON error objectTC-07Type"products" sent as an object instead of an arrayAdmin logged inPOST http://localhost:3000/api/admin/import-products{"products": {"name": "SP1", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}}Login admin@eshop.com/Admin123!400Content-Type: application/json; JSON error object indicating type validation failure (array expected)TC-08DomainProduct item missing</pre>

Name of the AI tool	Date and time	Your prompt	The AI output
		columns: Test ID Category Test Description Pre-conditions HTTP Method & URL Request Body Pre-request Script Expected HTTP Status Expected Response Body / JSON Schema Assertion Actual Result Verdict Rules for column values: Ensure every test case contains concrete values and payloads (no vague placeholders like "" or "<invalid_val>"). In "Pre-request Script": Specify login steps to acquire Admin Bearer token (admin@eshop.com / Admin123!) or regular User token (test@domain.com / Password123!) where applicable. In "Expected Response Body / JSON Schema Assertion": Do NOT hardcode exact error string assertions (e.g. avoid checking {"error": ""}). Only assert HTTP Status Codes, Content-Type: application/json, and positive schema/data assertions for 200 OK. Leave "Actual Result" and "Verdict" columns empty for test execution recording.	"name" fieldAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]}Login admin@eshop.com/Admin123!400 or 422Content-Type: application/json; JSON error objectTC-09DomainProduct item missing "price" fieldAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP1", "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]}Login admin@eshop.com/Admin123!400 or 422Content-Type: application/json; JSON error objectTC-10DomainProduct item missing "category_id" fieldAdmin logged inPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP1", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png"}]}Login admin@eshop.com/Admin123!400 or 422Content-Type: application/json; JSON error objectTC-11BoundaryProduct "name" as empty stringAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]}Login admin@eshop.com/Admin123!400 or 422Content-Type: application/json; JSON error objectTC-12TypeProduct "name" as numeric type instead of stringAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": 12345, "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]}Login admin@eshop.com/Admin123!400Content-Type: application/json; JSON error object indicating type validation failureTC-13BoundaryProduct "price" = 0Admin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP Free", "price": 0, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]}Login admin@eshop.com/Admin123!400 or 422Content-Type: application/json; JSON error object indicating price must be positive/non-zeroTC-14BoundaryProduct "price" = -1 (negative)Admin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP Negative", "price": -1, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]}Login admin@eshop.com/Admin123!400 or 422Content-Type: application/json; JSON error objectTC-15TypeProduct "price" as decimal/float valueAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP Float", "price": 99999.99, "description": "Mô tả", "image_id": "http://example.com/1.png", "category_id": 1}]}Login

[illegible]

Name of the AI tool	Date and time	Your prompt	The AI output
			http://localhost:3000/api/admin/import-products{"products": [{"name": "SP CatNull", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": null}]}Login admin@eshop.com/Admin123!400 or 422Content-Type: application/json; JSON error objectTC-23BoundaryProduct "name" extremely long string (5000 characters)Admin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "aaaaaaaaaa...(5000 'a' characters total)", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]}Login admin@eshop.com/Admin123!400 or 200 (document actual max-length behavior)Content-Type: application/json; response within 3s; NOT 500; if 200 confirm no silent truncation without errorTC-24TypeProduct "imageUrl" is a malformed non-URL stringAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP BadUrl", "price": 10000, "description": "Mô tả", "imageUrl": "not-a-valid-url", "category_id": 1}]}Login admin@eshop.com/Admin123!200 or 400 (spec shows imageUrl as free-form "http://..." example, format validation not explicitly documented)Content-Type: application/json; if 400, JSON error object; if 200, imageUrl stored exactly as submitted (confirms no format validation exists — flag as spec gap if unexpected)TC-25BoundaryProduct "imageUrl" field missing entirely (optional field per sample allowing empty "")Admin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP NoImage", "price": 10000, "description": "Mô tả", "category_id": 1}]}Login admin@eshop.com/Admin123!200 or 400 (document whether imageUrl is required — spec 6.3 sample shows "imageUrl": "" as acceptable, suggesting optional)Content-Type: application/json; if 200, imageUrl defaults to empty string or null; if 400, JSON error objectTC-26Business LogicDuplicate product names within the same batchAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP Duplicate", "price": 10000, "description": "Mô tả 1", "imageUrl": "http://example.com/1.png", "category_id": 1}, {"name": "SP Duplicate", "price": 20000, "description": "Mô tả 2", "imageUrl": "http://example.com/2.png", "category_id": 1}]}Login admin@eshop.com/Admin123!200 (spec does not document uniqueness constraint on name)Content-Type: application/json; imported count == 2 (both created as separate rows) — document actual behavior; flag as spec-gap item if a uniqueness rule is expected but undocumentedTC-27Business LogicBatch atomicity — one invalid item mixed with valid itemsAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products":

Name of the AI tool	Date and time	Your prompt	The AI output
			[{"name": "SP Valid1", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}, {"name": "SP Invalid", "price": -500, "description": "Mô tả", "imageUrl": "http://example.com/2.png", "category_id": 1}, {"name": "SP Valid2", "price": 20000, "description": "Mô tả", "imageUrl": "http://example.com/3.png", "category_id": 1}]]Login admin@eshop.com/Admin123!400 or 422 (all-or-nothing) OR 207/200 (partial success) — document actual atomicity modelContent-Type: application/json; if atomic: verify via GET /api/products?search=SP Valid1 that NEITHER valid item was persisted; if partial: response indicates which items succeeded/failed individuallyTC-28SecurityMissing Authorization header entirely—POST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP NoAuth", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]]No login performed; Authorization header deliberately omitted401Content-Type: application/json; JSON error objectTC-29SecurityInvalid/malformed Bearer token—POST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP BadToken", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]]Set header Authorization: Bearer garbage.invalid.token123 (no real login)401Content-Type: application/json; JSON error objectTC-30SecurityRegular (non-admin) user token attempts admin-only import (privilege escalation check)Regular user test@domain.com registered and logged inPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP UserAttempt", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]]Login test@domain.com/Password123! → store token as Authorization: Bearer 403Content-Type: application/json; JSON error object indicating insufficient permissions; verify product NOT created via GET /api/products?search=SP UserAttemptTC-31SecuritySQL injection payload in product "name" fieldAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP'; DROP TABLE products;--", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]]Login admin@eshop.com/Admin123!200 or 400 (must not corrupt data)Content-Type: application/json; if 200, subsequent GET /api/products still returns valid full product list (table not dropped); no DB error string in bodyTC-32SecurityXSS payload in product "description" fieldAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP XSS Test", "price": 10000, "description": "alert(document.cookie)", "imageUrl": "http://example.com/1.png", "category_id": 1}]]Login admin@eshop.com/Admin123!200 or 400Content-Type:

Name of the AI tool	Date and time	Your prompt	The AI output
			application/json; if 200, description stored/returned as inert JSON string, not executed HTML; Content-Type remains application/json not text/htmlTC-33RobustnessMalformed JSON syntax in request bodyAdmin logged inPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP Malformed", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1} (missing closing brackets — sent as raw malformed text)Login admin@eshop.com/Admin123!400Content-Type: application/json; JSON error object; NOT 500; no raw parser stack trace in bodyTC-34RobustnessInvalid Content-Type header (text/plain instead of application/json)Admin logged inPOST http://localhost:3000/api/admin/import-products{"products": [{"name": "SP WrongContentType", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1}]}Login admin@eshop.com/Admin123!; override request header to Content-Type: text/plain before sending400 or 415Content-Type: application/json; JSON error object indicating unsupported media type/parsing failureTC-35RobustnessMass assignment — extra unexpected fields injected into product objectAdmin logged in; category_id=1 existsPOST http://localhost:3000/api/admin/import-products{"products": [{"id": 999, "name": "SP MassAssign", "price": 10000, "description": "Mô tả", "imageUrl": "http://example.com/1.png", "category_id": 1, "is_admin": true, "created_at": "2000-01-01"}]}Login admin@eshop.com/Admin123!200 (with extra fields silently ignored)Content-Type: application/json; imported product's actual id is server-generated (NOT 999); no is_admin/created_at fields reflected into unrelated system state; verify via GET /api/products/:id that only documented schema fields are presentTC-36RobustnessHTTP verb tampering — GET method on import-products routeAdmin logged inGET http://localhost:3000/api/admin/import-products— (GET carries no body)Login admin@eshop.com/Admin123! (Authorization header set); no body sent404 or 405Content-Type: application/json; JSON error object; route/method not allowed Execution notes: TC-13/TC-14 (price ≤ 0) assume the business rule "positive, non-zero" stated in the task is enforced server-side — this is not explicitly confirmed in the spec text itself (spec only shows a positive sample value), so results should be recorded as a spec-gap defect if the API silently accepts zero/negative prices. TC-24/TC-25 (imageUrl format/optionality) are flagged as ambiguous by spec: Section 6.3's sample shows "imageUrl": "" as an acceptable value in an import context, suggesting the field may be optional/unvalidated — actual behavior should be confirmed with the dev team rather than assumed. TC-26 (duplicate names) and TC-27 (batch atomicity) are business-rule discovery tests — the spec does not document uniqueness constraints or transaction semantics for bulk import,

Name of the AI tool	Date and time	Your prompt	The AI output
			so both plausible outcomes are listed; whichever occurs should be documented as the confirmed behavior for regression baselining. TC-30 should be followed by confirming the regular user's own token still works correctly on permitted endpoints (e.g., GET /api/products) in the same session, to isolate that the 403 is role-specific and not a broader token failure. All SQL/XSS security tests (TC-31, TC-32) should include a post-test integrity check (GET /api/products, GET /api/categories) to confirm no destructive side effects occurred, per the same rationale used in the FR-06 and FR-08 suites. TC-36 can be extended with sibling DELETE/PUT verb-tampering cases if the team wants full method coverage; only GET is included here to stay within the fixed 36-case budget while still covering the verb-tampering dimension requested.

4. Conclusion — When should AI be used (or not)?

Overall, the AI model (Claude) demonstrated strong capabilities in automated API test generation, effectively synthesizing test suites across domain partitions, state transitions, security constraints, and schema validation. The majority of the generated test cases were valid, featuring well-structured payloads, deterministic assertions, and insightful edge cases. Nonetheless, quality degradation was observed at higher volumes of generation, occasionally yielding out-of-scope scenarios, ambiguous expected outcomes, or hallucinations of undocumented requirements. To mitigate these failure modes in future iterations, fine-grained prompting strategies—including explicit boundary constraints and negative prompting against requirement hallucinations—should be integrated into the audit and generation pipeline.

5. Mandatory Disclosure

The test suites were initially generated by Claude; I reviewed and modified TC03, TC04, TC05, TC33, TC34, TC35 in pool A suite, TC26, TC03, TC21, TC35 in pool B suite, TC23, TC29, TC36 in pool C suite; the main report, the bug report and the CI/CD report were written entirely by me. The detailed AI Audit Report is attached as Appendix A. I confirm I did not use AI to generate any artifact listed in the prohibited category.

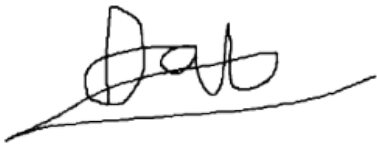
Signature

Student name (printed):	CAO TRẦN BÁ ĐẠT
Student ID:	23127168
Class / Cohort:	23KTPM1
Course:	CS423 / CSC13003 – Software Testing
Instructor:	LÂM QUANG VŨ
Date:	16/08/2026

**Student name
(printed):**

CAO TRẦN BÁ ĐẠT

Signature:



Cao Trần Bá Đạt

References

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- ISTQB Foundation Level Syllabus (latest version).
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building reliable AI test agents — engineering blog.
- DeepEval & Promptfoo documentation — testing frameworks for LLM systems.